Assignment Instructions

TODO.md

UI.md

API.md

Classroom_Survey_Web_App_Specification.md

DATABASE.md

CSE 4095 – Agentic Programming

Assignment: Build and Deploy a Classroom Survey Web App

Assignment Type: Individual
Duration: One week
Total Points: 100
Development Tools: Cursor or another AI-assisted coding environment

1. Overview

In this assignment, you will build and deploy a full-stack Classroom Survey Web App using an AI coding agent.

The application allows an instructor to create surveys for classroom use. Students can scan a QR code to access an active survey, submit answers without logging in, and have their responses stored in a database. The instructor can review survey results through a web interface.
You will be provided with software specification documents describing the application's requirements, database design, API, user interface, and implementation tasks.
Your primary goal is to use an AI coding agent to turn these specifications into a working, publicly deployed full-stack web application.
You are not expected to write every line of code manually. However, you are responsible for understanding the application architecture, configuring external services, testing the generated code, identifying problems, and guiding the AI agent to resolve them.

2. Learning Objectives

By completing this assignment, you should be able to:

1. Use an AI coding agent to implement a software application from written specifications.
2. Explain the roles of frontend, backend, and database components.
3. Create and configure a relational database using Supabase.
4. Understand how tables, primary keys, and foreign keys organize application data.
5. Connect a React application to a database.
6. Implement and test basic database operations.
7. Use GitHub for source code management.
8. Deploy a full-stack application using a cloud hosting service.
9. Debug application problems with assistance from an AI coding agent.

3. Provided Materials

You will receive the following five Markdown files:

File	Description
Classroom_Survey_Web_App_Specification.md	Overall application requirements
TODO.md	Implementation tasks and phases
DATABASE.md	Database schema and relationships
API.md	Application data access interfaces
UI.md	User interface and navigation

These files are the starting point for your implementation.
Read the documents before beginning. You may ask your AI coding agent to review them, identify ambiguities, and suggest an implementation plan.
You should follow the provided specifications. If you need to make a design decision that is not explicitly addressed, document your decision.

4. Functional Requirements

Your application must implement the following features.

4.1 Instructor Interface

The instructor should be able to:

- Log in using an instructor account.
- Create a new survey.
- Add multiple-choice and true/false questions.
- Edit existing draft surveys.
- View a list of surveys.
- Activate a survey for students.
- Close an active survey.
- View survey results.
- See response counts and percentages.
- Export results as CSV.

Only one survey should be active at a time.

4.2 Student Interface

Students should be able to:

- Access an active survey through a URL or QR code.
- View the survey questions.
- Select answers.
- Submit their responses.
- Receive confirmation after submitting.

Students should **not** need to create an account or log in.

The application should provide reasonable protection against accidental duplicate submissions.

4.3 Survey Presentation

The instructor should be able to display a presentation page containing:

- Survey title.
- QR code linking to the active survey.
- Survey URL.
- Number of submitted responses.

The presentation page should be suitable for classroom projection.

5. Database Requirements

The application must use a persistent relational database.

We recommend **Supabase (PostgreSQL)**, which provides a database, authentication, and APIs.

Your database should include the following tables, as described in DATABASE.md:

Table	**Purpose**
surveys	Store survey information
questions	Store questions belonging to surveys

choices	Store answer choices for questions
responses	Store individual student submissions
answers	Store answers associated with submissions

You must implement the relationships among these tables.

In particular:

- A survey can contain multiple questions.
- A question can contain multiple choices.
- A survey can receive multiple responses.
- A response can contain multiple answers.

Use appropriate primary keys and foreign keys to maintain data integrity.

Database security: Students must not be able to modify surveys or access instructor-only results. Use appropriate database access controls, including Supabase Row Level Security (RLS), as specified in the provided documents.

Database Verification

You must demonstrate that:

1. Creating a survey stores its information in the database.
2. Adding questions creates corresponding database records.
3. Student submissions are stored persistently.
4. Survey results can be retrieved from the database.
5. Data remains available after refreshing the application.

You should inspect the database using the Supabase Table Editor or SQL Editor.

6. Suggested Development Process

You are encouraged to follow the phases in TODO.md.

Phase 1: Review the specifications

Open the project in Cursor and provide the AI agent with the Markdown files.

An example initial prompt is:

Review all the Markdown specification files in this project. Explain the application architecture, identify any missing decisions, and propose an implementation plan. Do not start coding until you have reviewed the requirements.

Phase 2: Implement the frontend

Ask your AI agent to create the React application and implement the instructor and student interfaces.

Run the application locally and verify that the pages display correctly.

Phase 3: Configure the database

Create a Supabase project.

Use the provided database specification to create the required tables, relationships, and access policies.

Configure the application to connect to Supabase using environment variables.

Do not commit passwords, service-role keys, or other secrets to GitHub.

Phase 4: Implement database integration

Connect the application to Supabase.

Test the following workflow:

1. Instructor creates a survey.
2. Instructor adds questions and answer choices.
3. Instructor activates the survey.
4. Student opens the survey.
5. Student submits answers.
6. Responses are saved in the database.
7. Instructor views the results.

Phase 5: Test and debug

Test the application on both a computer and a mobile device.

Pay particular attention to QR-code access, student submissions, database persistence, and error handling.

If a problem occurs, use your AI coding agent to investigate and fix it.

Phase 6: Publish to GitHub

Create a GitHub repository for your project.

Commit and push your source code, specifications, and documentation.

Make sure sensitive configuration files are excluded.

Phase 7: Deploy the application

Deploy the application using Cloudflare Pages or another suitable hosting platform.

Verify that the public application works without requiring users to connect to your local network.

The database should remain hosted separately using Supabase or an equivalent service.

7. Required Deliverables

Submit the following through HuskyCT.

A. GitHub Repository URL

Your repository must contain:

- Application source code.
- Provided specification documents.
- Updated TODO.md indicating completed tasks.
- Database migration or schema files.
- README.md with setup and deployment instructions.

B. Deployed Application URL

Provide the public URL of your working application.

The instructor should be able to create surveys and students should be able to submit responses through the deployed application.

Provide a way for the instructor to test the instructor interface without publicly exposing credentials in the repository.

C. Development Summary

Submit a short report (approximately 1–2 pages) addressing:

1. How you used the specification documents to guide the AI agent.
2. Which parts of the application were generated by the AI agent.
3. How you configured the database.
4. How the frontend communicates with the database.
5. At least one problem you encountered and how you resolved it.
6. How you deployed the application.
7. What you learned about full-stack development.

D. AI Development History

Export your Cursor conversation history or provide a Markdown summary of your interactions with the AI coding agent.

The history should show how you guided the agent through implementation, testing, and debugging.

E. Database Evidence

Include screenshots showing:

- The database tables in Supabase.
- At least one survey with associated questions.
- Stored student responses.
- The survey results displayed in your application.

Do not include sensitive credentials or private student information.

8. Grading Rubric

Category	Points
Frontend functionality and usability	15
Instructor survey management	15
Student survey and QR-code functionality	15
Database schema and relationships	15
Database integration, persistence, and security	15
GitHub repository and deployment	10
Testing and debugging	5
Development summary and AI interaction history	10
Total	**100**

9. Important Guidelines

Use of AI: You are encouraged to use Cursor, ChatGPT, or another AI coding agent throughout the assignment. You may ask the AI to generate code, explain unfamiliar concepts, identify bugs, and suggest improvements.

Understanding your implementation: Although AI can generate the code, you are responsible for the final application. You should be able to explain the major components, especially the database schema and how data moves between the frontend and backend.

Testing: Do not assume generated code is correct simply because it compiles. Verify the complete application workflow.

Security: Do not expose private API keys, passwords, or other credentials in your GitHub repository.
Original work: Each student must create and deploy their own application. You may use the provided specifications and AI-generated code, but you must independently configure, test, and document your implementation.

10. Submission Checklist

Before submitting, verify that:

- The application runs locally.
- The instructor can create and activate surveys.
- Students can access surveys through QR codes.
- Students can submit answers without logging in.
- Responses are saved in the database.
- The instructor can view results.
- The database tables and relationships are implemented.
- The application is deployed publicly.
- The GitHub repository contains the required files.
- Sensitive credentials are not committed.
- The development summary and AI interaction history are included.
- Database screenshots are included.

11. Final Goal

By the end of this assignment, you should have a working, publicly deployed Classroom Survey Web App. More importantly, you should understand how an AI coding agent can assist with the complete software development process—from specifications and implementation to database configuration, testing, debugging, and deployment.

Submission

Click to add text

Word count: 0

Drag files here
Maximum size: 2,560 MB
or

Upload a file Upload from cloud

Save and Close Submit